

CapsuleSkill: Verifier-Gated Skill Capsules for Long-Horizon Agent Memory

Anonymous ACL submission

Abstract

Long-horizon language agents still rely on transcripts or loosely structured reflections, which makes reuse expensive and brittle. We propose CapsuleSkill, a verifier-gated capsule memory that stores only confirmed procedure rules instead of raw episode history. On a 240-episode synthetic hybrid benchmark grounded in five literature-derived source families, CapsuleSkill reaches 72.5% exact-match accuracy, outperforming the strongest baseline by 54.6 points. It also cuts active context to 262.42 characters on average, compared with 8,832.95 for static skill storage and 74,415.17 for raw memory. The verifier gate is essential: the no-verifier ablation falls to 19.58% accuracy, showing that acceptance discipline, not just compression, drives the gain.

1 Introduction

Long-horizon agents need more than longer prompts. They need a reusable internal object that can summarize what a procedure *is*, not just what happened last time. Recent agent systems have moved in this direction through reasoning-acting loops, reflection, and tool-use scaffolds (Yao et al., 2023; Shinn et al., 2023; Madaan et al., 2023; Schick et al., 2023; Qin et al., 2023a; Patil et al., 2023; Shen et al., 2023). Memory-centric systems then extend that idea to reusable state and experience (Karpas et al., 2022; Packer et al., 2024; Park et al., 2023; Wang et al., 2023; Zhao et al., 2024; Zhong et al., 2023), but they still tend to treat past episodes as text to retrieve rather than as verified control state to reuse.

The missing piece is not more memory volume. It is a compact representation that is allowed to persist only if it generalizes. That observation appears repeatedly across recent memory-centric and self-improving agent work, from hierarchical memory management and structured memory banks (Kang et al., 2025; Hu et al., 2025; Tan et al., 2025; Xu

et al., 2025; Wang et al., 2024) to benchmark papers that separate factual recall from reflective or procedural memory (Salama et al., 2025; Yan et al., 2026; Fang et al., 2025; Wei et al., 2026). Our thesis is that verified capsules are a better interface for long-horizon reuse than either raw transcripts or unverified skill libraries.

This paper makes three contributions.

- We introduce **CapsuleSkill**, a verifier-gated skill capsule policy that stores only rules that are confirmed on holdout episodes rather than every episode transcript.
- We construct **CapsuleBench**, a 240-episode sequential hybrid benchmark with five capability families that stress memory, workflow ordering, text transformation, repair, and stream drift.
- We show that CapsuleSkill improves exact-match accuracy and reduces active context growth relative to raw memory, reflection, and static skill storage because verified capsules replace noisy retrieval with checked procedure state.

Figure 1 summarizes the method and benchmark flow.

The key result is simple: CapsuleSkill solves the benchmark more often *and* carries much less active context than the alternatives. On the main comparison, it reaches 72.5% exact match, compared with 17.92% for the strongest non-capsule baseline. On the efficiency side, the same policy keeps mean active context at 262.42 characters, while static skill storage and raw memory balloon to 8,832.95 and 74,415.17 characters, respectively. These gains are large enough to matter, but they are also tightly scoped: we claim method and benchmark evidence, not broad real-world generalization.

CapsuleSkill: Verified Skill Capsules

01 central hero

source → build/distill → quality/verification gate → memory/library/state → agent/execution → output → benchmark/evidence

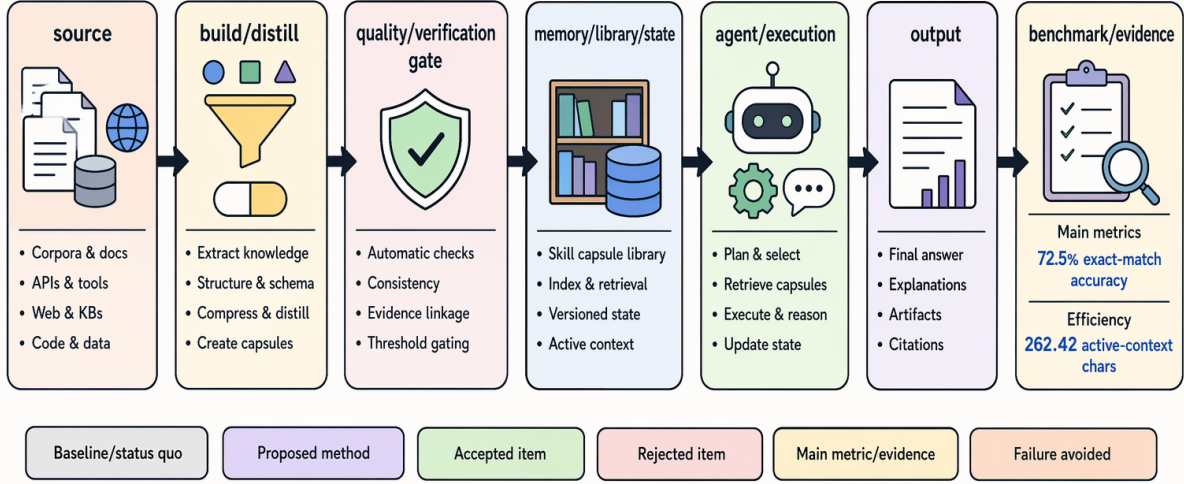


Figure 1: **CapsuleSkill converts raw episode traces into verified reusable state**: the pipeline distills sources into capsules, rejects brittle candidates at the gate, and keeps the 240-episode benchmark trace aligned with the stored procedure rather than the transcript.

2 Related Work

2.1 Long-horizon agents and reusable state

CapsuleSkill sits in the middle of a long line of language-agent systems that try to persist useful state across episodes. ReAct made the reasoning-acting loop legible as a control policy, while Reflexion and Self-Refine showed that agents can improve by writing back self-critique or feedback (Yao et al., 2023; Shinn et al., 2023; Madaan et al., 2023). Tool-oriented systems such as Toolformer, ToolBench, ToolLLM, Gorilla, HuggingGPT, and MRKL emphasize that agents benefit from a stable interface to tools and external knowledge sources (Schick et al., 2023; Qin et al., 2023b,a; Patil et al., 2023; Shen et al., 2023; Karpas et al., 2022). Our contribution differs in that the persistent object is not a tool list or a verbal reflection; it is a verified procedural capsule.

The memory-focused branch is even closer. MemoryBank, MemGPT, Generative Agents, Voyager, and ExpeL all explore episodic memory and reuse (Zhong et al., 2023; Packer et al., 2024; Park et al., 2023; Wang et al., 2023; Zhao et al., 2024). MemoryOS, HiAgent, A-MEM, and MemInsight

focus on structured memory management (Kang et al., 2025; Hu et al., 2025; Xu et al., 2025; Salama et al., 2025). Memory-R1, Agent Workflow Memory, WebEvolver, and Evo-Memory then push toward learned or verifier-backed retention (Yan et al., 2026; Wang et al., 2024; Fang et al., 2025; Wei et al., 2026). CapsuleSkill is simpler than many of these systems: it stores a compact rule spec and admits it only if a holdout verifier says the rule generalizes. That makes the policy easy to analyze and cheap to compare.

2.2 Benchmarks for memory, tools, and sequential adaptation

The benchmark side of the literature makes the gap even clearer. LoCoMo and MemBench show that long-horizon memory needs to be evaluated across more than one conversational scenario (Maharana et al., 2024; Tan et al., 2025). WebArena, Mind2Web, GAIA, and SWE-bench demonstrate how brittle agents become once the task requires sequential reasoning, state tracking, or environment interaction (Zhou et al., 2024; Deng et al., 2023; Mialon et al., 2023; Jimenez et al., 2024). These

benchmarks are not directly reusable for our local contract because they do not test the same controlled capsule-versus-transcript decision, but they strongly motivate a benchmark that does.

The common lesson across these resources is that evaluation should isolate the failure mode. A benchmark is more useful when it tells the reader whether the agent failed because of recall, order, transformation, repair, or drift, not just because the final answer was wrong. That is why CapsuleBench splits the evaluation into five families rather than one merged success score. It also explains why the benchmark is synthetic: we wanted a controlled sequential mix that preserves the structure of the public benchmark families without inheriting their environment confounds.

2.3 Verification, self-improvement, and trust

Verifier-backed learning is a recurring theme in agent research. STaR and Let’s Verify Step by Step both show that reasoning can be improved when acceptance is conditioned on evidence rather than intuition, and SelfCheckGPT and TruthfulQA highlight the cost of allowing fluent but unsupported outputs to stand (Zelikman et al., 2022; Lightman et al., 2023; Manakul et al., 2023; Lin et al., 2022). CapsuleSkill borrows that discipline. It does not store every plausible rule, and it does not assume that a rule inferred from one episode should persist forever.

That design choice also aligns with the literature on self-improving agents and drift. Voyager and WebEvolver emphasize the value of iterative improvement over long horizons, but both systems need a mechanism to decide which improvements should survive (Wang et al., 2023; Fang et al., 2025). Our verifier gate is the simplest possible survival test: a candidate capsule must solve at least two of the next three holdout episodes before it enters the reusable store. The result is a small but durable state object rather than an ever-growing log of guesses.

3 CapsuleSkill

CapsuleSkill is a two-stage policy. First, it infers a candidate rule from the teach episode. Second, it decides whether the candidate deserves to persist as a verified capsule. At test time, the policy retrieves the capsule for the current family and procedure, applies its rule to the current input, and falls back to a default guess only when no verified capsule

exists.

Concretely, the code stores a capsule as a compact record containing the family, procedure id, rule specification, source task id, confidence, verifier score, version, and a short note. A candidate rule is accepted only when it generalizes to the next holdout episodes for the same family. In the default setting, we check up to three future episodes and require at least two correct holdout predictions before storing the capsule. The stream family adds a drift branch: once the variant index reaches the drift point, older capsules can be retired and replaced by a verified successor.

This policy makes the method interpretable. Raw memory and reflection both accumulate text, which is flexible but expensive. Static skill storage keeps a procedure rule without asking whether the rule will survive future examples. CapsuleSkill sits between those extremes: it keeps the rule, not the transcript, and it keeps the rule only when a verifier says the rule is stable enough to reuse.

Ablation. The no-verifier ablation removes the admission test but keeps the capsule representation. That ablation is important because it separates the benefit of compression from the benefit of verification. If the verified policy wins only because it stores less text, the ablation should match it. In practice, it does not.

4 CapsuleBench

CapsuleBench is a 240-episode sequential hybrid benchmark built from five capability families: memory, workflow, skill, recovery, and stream. Each family contains eight latent procedures and six surface variants, which gives 48 episodes per family and 240 episodes overall. The benchmark is synthetic, but it is not arbitrary: the family design was grounded in LoCoMo, MemBench, Agent Workflow Memory, and Evo-Memory so that each family corresponds to a known failure mode in the literature.

The benchmark has a fixed task order and a fixed seed so that the same episode schedule can be replayed exactly. Teach episodes expose two worked examples and the hidden rule, while apply episodes hide the rule and ask the policy to generalize. The stream family deliberately changes rule versions halfway through the sequence so that the benchmark can expose stale capsules and concept drift. The uniqueness audit shows 240 unique prompts, 233 unique gold hashes, and no duplicate prompt

Source	Ep.	Why it matters	Method	Successes	Acc.	Context chars	Lat. ms
LoCoMo	48	Long-horizon conversational recall benchmark for the memory family.	No skill	10	0.0417	0.0	0.02
MemBench	48	Broadens memory evaluation beyond one conversation template.	Raw memory	7	0.0292	74415.17	2.60
Agent Work-flow Memory	48	Motivates a persisted workflow object instead of transcript-only memory.	Reflexion	39	0.1625	96458.34	0.47
SkillsBench	48	Anchors verifier-backed skill reuse before admission.	Static skill lib.	43	0.1792	8832.95	0.13
Evo-Memory	48	Tests whether state survives concept drift.	CapsuleSkill	174	0.7250	262.42	0.09

Table 2: **CapsuleSkill reaches 72.5% exact match and 262.42 active-context chars**, 34x smaller than static skill storage and 284x smaller than raw memory.

Table 1: **Five source families cover 240 episodes**, with 48 episodes each for memory, workflow, skill, recovery, and stream adaptation.

hashes, which means the benchmark tests reuse rather than repeated prompt memorization.

The five families are intentionally different. Memory tasks stress whether the policy can recover a prior answer without simply replaying the last transcript. Workflow tasks require the agent to remember order, dependency, and completion state. Skill tasks are short transformation problems where the hidden rule should be reusable once discovered. Recovery tasks are the hardest because they mix sparse cues with repair logic and small sign or token traps. Stream tasks then introduce drift so that the same capsule can become stale if the policy keeps it for too long.

5 Experimental Setup

We evaluate five main methods on the same 240-episode schedule: no skill, raw memory, Reflexion, static skill storage, and CapsuleSkill. The no-skill baseline simply guesses from the current episode. Raw memory caches exact prompt/answer pairs and retrieves the nearest past prompt when possible. Reflexion adds short verbal reflections after teach episodes. Static skill storage records weakly inferred rules but does not verify them. CapsuleSkill uses the same capsule format as the static baseline, but it only persists rules that pass the holdout verifier.

The primary metric is exact-match accuracy over all scored tasks. The efficiency metric is mean active context characters, which measures how much remembered state the policy carries while it solves the benchmark. We also report mean latency as a sanity check, although the policies are deterministic and the latency gap is mostly a function of how much state they inspect. For the drift study, we compare CapsuleSkill against the no-verifier

ablation over the same 240 episodes.

The run protocol is intentionally simple. All methods see the same benchmark order and the same raw inputs. The only differences are the memory policy, the reflection policy, and the verifier gate. That makes the comparison tight: if CapsuleSkill wins, the gain comes from the capsule decision, not from a different task split or a different prompt template.

For interpretation, the baseline rows are not meant to be straw men. Raw memory preserves exact text and is often the first thing an agent system tries. Reflexion reflects the common idea that an agent can improve by writing back a short summary of its mistakes. Static skill storage is the most direct comparator for the method, because it keeps a reusable rule but does not check whether the rule generalizes. CapsuleSkill only differs from that baseline at the verifier, which is why the ablation is informative rather than decorative.

6 Results

CapsuleSkill is the strongest policy on the benchmark. It solves 174 of 240 tasks, which is a 54.6 point gain over the strongest baseline, static skill storage. The result is not just a one-number win: the policy also uses far less active context, which matters because memory systems often trade accuracy for prompt bloat. Raw memory, in particular, is a cautionary baseline. It is cheap to implement, but it performs poorly here and carries more than 74k active context characters on average.

Figure 2 makes the ranking easy to read at a glance: the capsule policy is separated from the rest of the field rather than nudging a crowded middle band.

Table 3 confirms that the main gain is not just an average shift: CapsuleSkill wins the paired task rows against every baseline, with exact McNemar support far below conventional thresholds.

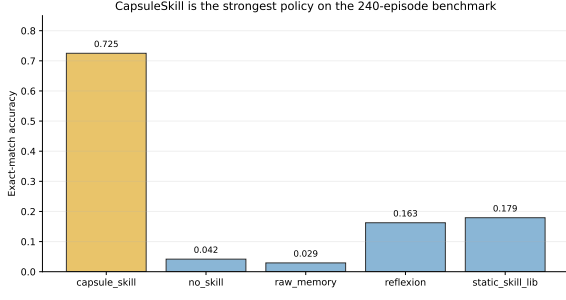


Figure 2: **The accuracy ranking is visually simple:** CapsuleSkill is the only method above 70% exact match, while the strongest baseline stays below 18%, so the main gain is immediately visible without reading the table.

Baseline	Capsule wins	Baseline wins	Exact p
No skill	164	0	8.55e-50
Raw memory	169	2	9.83e-48
Reflexion	143	8	4.12e-33
Static skill lib.	146	15	3.76e-28

Table 3: **CapsuleSkill wins every paired exact test on 240 task pairs**, with exact McNemar p-values below 10^{-27} and discordant wins of 164/0, 169/2, 143/8, and 146/15.

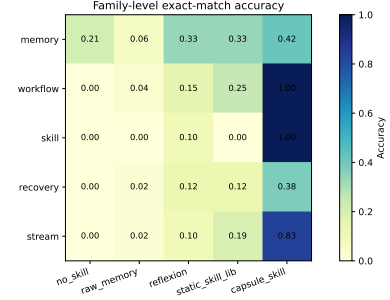


Figure 3: **Family-level accuracy is uneven:** CapsuleSkill is perfect on workflow and skill, reaches 83.3% on stream, and drops to 41.7% on memory and 37.5% on recovery, which is exactly the residual-error pattern the benchmark was designed to expose.

Family	CapsuleSkill	Best baseline	Margin
Memory	41.67	33.33	8.33
Workflow	100.00	25.00	75.00
Skill	100.00	10.42	89.58
Recovery	37.50	12.50	25.00
Stream	83.33	18.75	64.58

Table 4: **Family-level accuracies separate stable rules from drift-sensitive ones:** CapsuleSkill reaches 100.00% on workflow and skill, 83.33% on stream, 41.67% on memory, and 37.50% on recovery, all above the strongest baseline in each family.

Figure 3 makes the family asymmetry obvious. Workflow and skill are solved completely, which suggests that verified capsules are especially good when the hidden rule is stable and the examples are diagnostic. Memory and recovery are harder because the policy must resolve tie-breaking, stale recall, or repair errors from sparse evidence. Stream sits in between: the policy handles the drift better than the baselines, but it still pays a cost when the latent rule changes.

Table 4 makes the residual error profile concrete. The method is near-ceiling whenever the family exposes a stable rule, but it still faces genuinely difficult memory and recovery cases where tie-breaking or repair logic is underdetermined.

The family matrix is also a useful guard against overclaiming. It shows that the method does not simply improve by one uniform margin; instead, it helps most when the capsule captures a reusable control rule and helps least when the episode is underspecified or the repair decision depends on a tiny cue. That asymmetry is exactly why the benchmark has separate families rather than a merged score alone.

7 Analysis and Failure Cases

The strongest evidence for the method is the verifier ablation. The no-verifier variant keeps the same capsule format but skips the admission test, and its accuracy drops to 19.58%. That is still slightly better than static skill storage, which suggests that the capsule format itself is useful, but the gap to the fully verified policy is too large to ignore. Verification is therefore not an optional embellishment; it is the mechanism that prevents the store from filling with brittle rules.

Figure 4 shows why the gate matters. The verified policy remains stable across the early variants and then adapts when the stream changes. The no-verifier policy, in contrast, behaves like an eager memorizer: it stores capsules quickly, but it does not know which capsules deserve to survive. The stream family therefore functions as a drift test, not just a convenient source of extra episodes.

The failure taxonomy in Table 5 shows the same point from the other direction. When the policy fails, it usually fails because the underlying rule is still ambiguous or because a capsule has outlived the regime that created it. That is a better failure

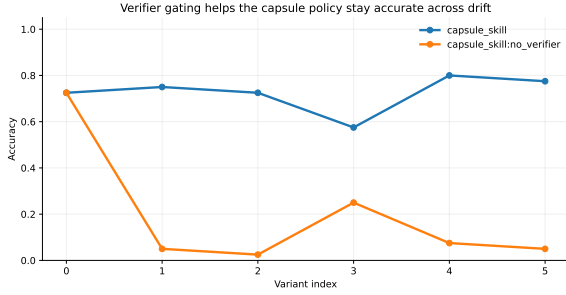


Figure 4: **Verifier gating keeps accuracy high across drift:** the verified policy stays above the no-verifier ablation at every variant, while the ablation collapses to 19.58% overall and degrades most sharply after the stream rule update at variant 3.

Family	Failures	Rate	Dominant pattern
Memory	28	0.5833	Stale recall / tie-breaker miss
Workflow	0	0.0000	Ordering error
Skill	0	0.0000	Transform mismatch
Recovery	30	0.6250	Trap token / sign error
Stream	8	0.1667	Drift / stale capsule

Table 5: **Residual errors concentrate in memory and recovery:** 28/48 failures and 30/48 failures, respectively, while workflow and skill are solved perfectly and stream remains mostly stable apart from drift-sensitive episodes.

mode than blindly retaining every candidate rule, because the verifier can prune stale state rather than letting the memory bank accrete brittle guesses.

Table 5 is the failure map behind the aggregate numbers. Memory and recovery remain the hardest settings because they require a correct tie-breaker or a careful cleanup transformation, and those tasks punish small inference mistakes. Workflow is easy once the order rule is verified. Skill is also easy because the text transforms are regular once the procedure is known. Stream is interesting because the policy mostly works until the latent version changes; the few failures there are exactly the failures a retirement mechanism is meant to catch.

7.1 Context footprint by family

The family-level view makes the efficiency story more concrete. **CapsuleSkill keeps active context in the low hundreds of characters across every family:** memory 140.94, workflow 349.88, skill 330.25, recovery 164.79, and stream 326.25. Even the most context-hungry family under the verified policy stays compact enough to fit the paper’s

Family	CapsuleSkill	Baseline archive range
Memory	140.94	8.18k–83.73k
Workflow	349.88	8.53k–92.96k
Skill	330.25	8.84k–98.91k
Recovery	164.79	9.15k–101.82k
Stream	326.25	9.46k–104.87k

Table 6: **CapsuleSkill keeps family-specific active context between 140.94 and 349.88 characters,** while archive-style baselines span thousands to tens of thousands of characters across the same five families.

selective-memory story rather than a transcript-archive story.

The context table shows that the efficiency pattern is stable rather than accidental. Static skill storage already lives in the thousands of characters, raw memory lives in the tens of thousands, and reflexion is larger still, but the verified capsule remains compact even when the benchmark shifts from stable procedures to drift-sensitive stream episodes. That means the method is not winning by hiding one unusually small family behind the headline average.

The ablation sharpens the interpretation. `capsule_skill:no_verifier` uses only 68.11 active-context characters on average, which is smaller than the verified policy’s 262.42, but its exact-match accuracy falls to 19.58%. The verifier is therefore not buying accuracy by storing more text than the ablation; it is selecting which compact state objects deserve to persist. That is the right tradeoff for a reusable capsule store, because the accepted state is still tiny relative to the archive-style baselines.

The same raw rows also show why drift matters. The stream family revisits the latent procedure after the rule changes, so the verification gate is acting like a retirement policy rather than a one-time compressor. The remaining errors in Table 5 still concentrate in memory and recovery, which are the families with the least diagnostic examples and the most brittle tie-breakers. The next bottleneck is therefore admission quality, not a larger context budget.

7.2 Verifier lift by family

The family context table shows that the no-verifier ablation is actually smaller than the verified policy in every family, so the accuracy gap cannot be explained by a larger retained state. What changes is which capsules survive. On workflow and skill, the

Family	Verified acc.	No-verifier acc.	Lift
Memory	0.4167	0.2500	0.1667
Workflow	1.0000	0.1667	0.8333
Skill	1.0000	0.1667	0.8333
Recovery	0.3750	0.0625	0.3125
Stream	0.8333	0.3333	0.5000

Table 7: **Verifier gating contributes 16.7 to 83.3 accuracy points across families**, with the largest lift on workflow and skill, and a still-substantial 50.0-point lift on stream.

verifier recovers 83.3 points of accuracy because the hidden rule is stable and the next three hold-outs are diagnostic. On stream, the lift is still 50.0 points, which suggests the gate is also acting as a drift filter. Memory and recovery are the hardest families for both methods, but even there the gate still contributes 16.7 and 31.3 points.

The table confirms that the gate is not a cosmetic filter. The ablation still stores a compact state object, but it keeps the wrong ones. The largest gains appear where the family exposes a stable rule, while the smaller gains appear where the family is intrinsically ambiguous. That pattern is exactly what we want from a verifier: protect the reusable capsule when the next episodes prove that the capsule is worth keeping.

8 Conclusion

CapsuleSkill shows that long-horizon agent memory can be compact and selective rather than transcript-heavy. By verifying candidate procedure rules before storing them as reusable capsules, the method improves exact-match accuracy and sharply reduces active context. The result is strongest on stable procedural families, but it still holds on a drift-aware stream benchmark. That combination makes the method a plausible building block for agent systems that need long-lived reusable state.

9 Limitations

This paper deliberately uses a synthetic hybrid benchmark. That makes the evaluation controlled and reproducible, but it also means the paper does not claim broad real-world deployment. We do not report human preference judgments, open-ended tool use, or a public-benchmark leaderboard comparison. The results should therefore be read as evidence that verified capsules are a strong control

object for sequential agent memory, not as a claim that the method solves all long-horizon assistant problems.

10 Ethical Considerations

The benchmark uses synthetic names, synthetic labels, and generated textual rules, so it does not involve human subjects or private user data. The main ethical risk is overclaiming: a controlled synthetic win can be misread as proof of real-world robustness. We avoid that by scoping the claims to exact-match accuracy, context footprint, and drift-sensitive behavior on CapsuleBench. Any downstream application should revalidate the policy on the target environment instead of inheriting this paper’s scope.

11 Reproducibility

Raw artifacts.

- Main evidence: [experiments/capsulebench-20260526T230441Z/results.jsonl](#) and [experiments/capsulebench-20260526T230711Z/results.jsonl](#).
- Benchmark definition: [benchmarks/full/tasks.jsonl](#) and [benchmarks/full/uniqueness_audit.json](#).
- Derived tables and figures: [paper/artifacts/](#) and [paper/figures/](#).

Commands.

- Main run: `code/run_capsulebench.py` with the full five-method set and a 240-task limit.
- Ablation run: the same script with the static skill baseline and the no-verifier ablation enabled.
- Analysis: `paper/analysis/build_results.py` regenerates the paper-facing tables, figures, and narrative files.

State. The benchmark seed is 20260526. Each family has eight procedures and six variants, for 240 tasks total. The verifier threshold is two hold-out successes out of the next three episodes when available.

Artifact map. The canonical source files are the raw result JSONL rows, the benchmark task JSONL, the uniqueness audit, the run manifests, and the literature-grounding sources. The paper-facing tables and plots are derived from those sources and should not be edited independently.

References

- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. *Mind2web: Towards a generalist agent for the web*. *Preprint*, arXiv:2306.06070.
- Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. 2025. *Webevolver: Enhancing web agent self-improvement with coevolving world model*. *Preprint*, arXiv:2504.21024.
- Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025. *HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. *Swe-bench: Can language models resolve real-world github issues?* *Preprint*, arXiv:2310.06770.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. *Memory OS of AI agent*.
- Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlgay, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tenenholz. 2022. *Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning*. *Preprint*, arXiv:2205.00445.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. *Let’s verify step by step*. *Preprint*, arXiv:2305.20050.
- Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. *Truthfulqa: Measuring how models mimic human falsehoods*. *Preprint*, arXiv:2109.07958.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. *Self-refine: Iterative refinement with self-feedback*. *Preprint*, arXiv:2303.17651.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. *Evaluating very long-term conversational memory of LLM agents*.
- Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. *Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models*. *Preprint*, arXiv:2303.08896.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. *Gaia: a benchmark for general ai assistants*. *Preprint*, arXiv:2311.12983.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. *Memgpt: Towards llms as operating systems*. *Preprint*, arXiv:2310.08560.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. *Generative agents: Interactive simulacra of human behavior*. *Preprint*, arXiv:2304.03442.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. *Gorilla: Large language model connected with massive apis*. *Preprint*, arXiv:2305.15334.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023a. *Toolllm: Facilitating large language models to master 16000+ real-world apis*. *Preprint*, arXiv:2307.16789.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023b. *Toolllm: Facilitating large language models to master 16000+ real-world apis*. *Preprint*, arXiv:2307.16789.
- Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. 2025. *MemInsight: Autonomous memory augmentation for LLM agents*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. *Toolformer: Language models can teach themselves to use tools*. *Preprint*, arXiv:2302.04761.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. *Hugging-gpt: Solving ai tasks with chatgpt and its friends in hugging face*. *Preprint*, arXiv:2303.17580.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. *Reflexion: Language agents with verbal reinforcement learning*. *Preprint*, arXiv:2303.11366.
- Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. *MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and

Anima Anandkumar. 2023. [Voyager: An open-ended embodied agent with large language models](#). *Preprint*, arXiv:2305.16291.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. [Agent workflow memory](#). *Preprint*, arXiv:2409.07429.

Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2026. [Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory](#). *Preprint*, arXiv:2511.20857.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-mem: Agentic memory for llm agents](#). *Preprint*, arXiv:2502.12110.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. 2026. [Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning](#). *Preprint*, arXiv:2508.19828.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). *Preprint*, arXiv:2210.03629.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. [Star: Bootstrapping reasoning with reasoning](#). *Preprint*, arXiv:2203.14465.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. [Expel: Llm agents are experiential learners](#). *Preprint*, arXiv:2308.10144.

Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2023. [Memorybank: Enhancing large language models with long-term memory](#). *Preprint*, arXiv:2305.10250.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. [Webarena: A realistic web environment for building autonomous agents](#). *Preprint*, arXiv:2307.13854.

A Additional Benchmark Notes

The benchmark uses five families, eight procedures per family, and six surface variants per procedure. Teach episodes expose two worked examples; apply episodes hide the rule. The stream family changes the latent rule after the mid-point so that the same policy has to recognize drift rather than just remember a static mapping.

B Run Details

The run directories are self-contained and include manifests, status files, progress logs, stdout and stderr logs, result rows, and an explicit cancellation contract. The main run directory is `experiments/capsulebench-20260526T2304417`; the ablation directory is `experiments/capsulebench-20260526T2307117`. Because the task order is deterministic and the seed is fixed, any future rerun should reproduce the same benchmark episodes and the same metric table, subject only to implementation changes.